

PSL Recognition — 3DCNN-ResBiLSTM Signer-Disjoint Retrain Report

Experiment 1

Run: [logs](#) | 50 epochs | Best checkpoint: epoch 43 (val acc 10.45%)

1. Model architecture

_3DCNN_LSTM_Model_Modified — 3D CNN feature extractor + bidirectional LSTM temporal head.

Stage	Layer	Output
conv3d_1	Conv3d(3→32, k3, stride (1,2,2)) + BN + ReLU + MaxPool3d(1,2,2)	downsample spatial only
conv3d_2	Conv3d(32→64, k3) + BN + ReLU + MaxPool3d(2,2,2)	downsample spatiotemporal
conv3d_3	Conv3d(64→128, k3) + BN + ReLU	
res_block1	ResidualBlock(128): 2×(Conv3d+BN), skip-add, ReLU	
pool3	MaxPool3d(2,2,2)	
res_block2	ResidualBlock(128)	
pool4	MaxPool3d(2,2,2)	flatten to (B, T, 1152)
normalize	per-timestep L2 normalize (dim=2)	BatchNorm1d removed — see D4
LSTM	BiLSTM, hidden=576, 2 layers, dropout 0.3	(B, T, 1152)

readout mean-pool over all timesteps (not last-step) see D2

head Dropout(0.3) → Linear(1152 → 104) logits

Input: 120 frames/clip, 112×112, RGB, uniform `np.linspace` temporal sampling (D3). No zero-padding.

Key fixes vs. prior version (decision log)

- **D1** — Test evaluated on originals only (312 clips); augmented test copies excluded to avoid inflated/correlated sample counts.
- **D2** — Removed zero-padding + last-timestep readout (was feeding the backward LSTM a black-frame-derived vector). Replaced with uniform sampling + full-sequence mean pooling.
- **D3** — `np.linspace(0, N-1, num_frames)` uniform temporal sampling regardless of clip length.
- **D4** — Dropped BatchNorm1d before LSTM (was fighting the following L2 normalization); kept per-sample L2 norm.
- **D6** — Stage-5 training recipe: AdamW (lr 1e-4, wd 1e-3), label smoothing 0.04, grad clip 1.0, cosine LR + 3-epoch linear warmup, dropout 0.5→0.3, batch 16→32, real early stopping on val acc (patience 15) with best-checkpoint (not latest) saved.
- **D8** — Pre-extracted `.npy` full-frame caches for train/val (removes cv2 decode-per-epoch bottleneck); test stays on raw video decode.
- **D9** — Top-1/3/5 accuracy on test, cross-checked against argmax accuracy (hard assert).

Compute: 11.9–12.2 GB VRAM. Run crossed 6 Colab session boundaries (restarts at epochs 5, 23, 30, 33, 33, 36) — resume/checkpoint logic held, no lost progress

2. Per-epoch metrics (CSV)

epoch	epoch_time_sec	lr	train_loss	train_acc	train_prec	train_recall	train_f1	eval_loss	eval_acc	eval_prec	eval_recall	eval_f1	is_best	epoch_would_improve
1	4473.87	5.036496350364964e-05	4.6451	1.03	0.0007	0.0103	0.0011	4.6445	0.96	0.0001	0.0096	0.0002	True	0
2	3635.32	0.0001	4.6432	0.96	0.0066	0.0096	0.0036	4.6185	1.85	0.0016	0.0185	0.0026	True	0
3	3453.72	9.989294616193017e-05	4.429	1.6	0.012	0.016	0.0076	4.3559	2.49	0.0087	0.0249	0.0065	True	0
4	3360.61	9.957224306869053e-05	4.22	2.47	0.0161	0.0247	0.015	4.2341	2.73	0.0031	0.0273	0.0053	True	0
5	3294.97	9.903926402016153e-05	4.0687	3.41	0.0207	0.0341	0.0205	4.1451	3.7	0.0105	0.037	0.012	True	0
5	2522.91	9.903926402016153e-05	4.0687	3.21	0.0237	0.0321	0.0202	4.1896	3.54	0.0117	0.0354	0.0123	True	0
6	2238.89	9.829629131445342e-05	3.9514	4.14	0.0347	0.0414	0.028	4.1163	2.57	0.0062	0.0257	0.0072	False	1
7	2055.56	9.73465064747553e-05	3.811	6.89	0.0565	0.0689	0.0479	4.0717	4.66	0.0064	0.0466	0.011	True	0
8	2021.61	9.619397662556435e-05	3.6948	7.3	0.0645	0.073	0.058	4.0275	3.78	0.0131	0.0378	0.0165	False	1
9	1963.59	9.484363707663442e-05	3.6362	6.3	0.0572	0.063	0.0476	3.9914	5.14	0.0235	0.0514	0.0268	True	0
10	1926.98	9.330127018922194e-05	3.5563	7.65	0.0577	0.0765	0.0588	3.9603	4.42	0.0263	0.0442	0.0262	False	1
11	1917.25	9.157348061512727e-05	3.4493	8.61	0.0747	0.0861	0.071	3.9367	4.5	0.0281	0.045	0.0235	False	2
12	1913.97	8.966766701456177e-05	3.3616	9.46	0.0926	0.0946	0.0789	3.9418	4.18	0.0186	0.0418	0.0229	False	3
13	1981.87	8.759199037394887e-05	3.2706	11.33	0.0951	0.1133	0.0943	3.9308	6.43	0.0342	0.0643	0.039	True	0
14	1950.84	8.535533905932738e-05	3.1531	13.1	0.1281	0.131	0.1105	3.8918	6.27	0.0531	0.0627	0.0467	False	1
15	1929.5	8.296729075500344e-05	3.0581	15.25	0.1489	0.1525	0.1325	3.9704	6.35	0.0394	0.0635	0.0402	False	2
16	1907.26	8.043807145043604e-05	2.9843	16.46	0.1581	0.1646	0.1441	3.9326	6.35	0.0359	0.0635	0.043	False	3
17	1975.52	7.777851165098012e-05	2.8672	19.76	0.2065	0.1976	0.1772	3.8912	6.43	0.042	0.0643	0.0464	False	4
18	1951.58	7.5000000000000001e-05	2.7783	20.97	0.1987	0.2097	0.188	3.9576	8.36	0.0696	0.0836	0.0633	True	0
19	2051.16	7.211443451095007e-05	2.6352	25.14	0.2535	0.2514	0.2282	3.938	7.23	0.0693	0.0723	0.0586	False	1

20	2022.64	6.91341716182545e-05	2.5622	27.29	0.2708	0.2729	0.2528	3.9997	6.75	0.0676	0.0675	0.0539	False	2
21	2108.22	6.607197326515808e-05	2.4665	29.97	0.2863	0.2997	0.278	4.0162	6.67	0.0636	0.0667	0.0549	False	3
22	1996.97	6.294095225512603e-05	2.3718	33.04	0.3236	0.3304	0.3066	4.0505	7.23	0.0635	0.0723	0.06	False	4
23	2009.03	5.9754516100806423e-05	2.2596	35.62	0.347	0.3562	0.3346	4.022	7.4	0.0656	0.074	0.062	False	5
23	4563.54	5.9754516100806423e-05	2.2289	39.08	0.3787	0.3908	0.3679	4.0567	7.32	0.0669	0.0732	0.0643	False	4
24	3630.57	5.6526309611002594e-05	2.139	41.19	0.4041	0.4119	0.3924	4.1972	7.4	0.0679	0.074	0.0661	False	4
25	3371.92	5.327015646150716e-05	2.009	47.32	0.472	0.4732	0.4556	4.2321	8.44	0.0902	0.0844	0.0761	True	0
26	3295.74	5e-05	1.9234	49.54	0.4878	0.4954	0.4776	4.1954	7.96	0.0698	0.0796	0.0667	False	0
27	3288.12	4.6729843538492847e-05	1.8525	51.05	0.5041	0.5105	0.4911	4.2053	8.76	0.086	0.0876	0.0796	True	0
28	3305.29	4.347369038899744e-05	1.749	55.56	0.5508	0.5556	0.541	4.3373	9.08	0.0954	0.0908	0.0842	True	0
29	3247.46	4.0245483899193595e-05	1.6882	58.04	0.5765	0.5804	0.5663	4.3195	9.89	0.0945	0.0989	0.0893	True	0
30	3286.65	3.705904774487396e-05	1.5889	61.93	0.6168	0.6193	0.6045	4.3771	9.08	0.0955	0.0908	0.0851	False	0
31	2563.89	3.392802673484193e-05	1.4918	66.53	0.6649	0.6653	0.6536	4.3876	9.32	0.0888	0.0932	0.0852	False	0
32	2170.63	3.086582838174551e-05	1.4198	69.48	0.6975	0.6948	0.6865	4.4369	9.08	0.0897	0.0908	0.0845	False	0
33	2174.28	2.7885565489049946e-05	1.3283	74.59	0.7474	0.7459	0.7392	4.4645	10.29	0.0988	0.1029	0.0928	True	0
33	4419.06	2.7885565489049946e-05	1.3271	74.11	0.7432	0.7411	0.7338	4.4469	9.32	0.0926	0.0932	0.0873	False	0
34	3429.29	2.5000000000000001e-05	1.2556	77.24	0.7751	0.7724	0.7673	4.5063	9.08	0.1004	0.0908	0.0862	False	0
35	3281.29	2.2221488349019903e-05	1.207	78.55	0.7875	0.7855	0.7807	4.5484	9.81	0.0912	0.0981	0.0886	False	0
36	3222.3	1.9561928549563968e-05	1.1612	80.65	0.8081	0.8065	0.8027	4.5968	9.81	0.0986	0.0981	0.0925	False	0
37	2777.39	1.703270924499656e-05	1.0882	83.75	0.8438	0.8375	0.8347	4.572	9.81	0.1022	0.0981	0.0933	False	0
38	2483.75	1.4644660940672627e-05	1.0451	85.21	0.8555	0.8521	0.8495	4.6221	9.49	0.0912	0.0949	0.088	False	0
39	2422.64	1.2408009626051137e-05	1.0073	87.02	0.874	0.8702	0.8674	4.6714	9.08	0.098	0.0908	0.0878	False	0
40	2328.83	1.0332332985438248e-05	0.9803	87.75	0.8794	0.8775	0.8747	4.6816	9.81	0.1004	0.0981	0.0923	False	0
41	2415.9	8.426519384872733e-06	0.9705	87.68	0.8805	0.8768	0.8749	4.6964	9.73	0.1054	0.0973	0.0935	False	0

42	2349.44	6.698729810778065e-06	0.9343	88.83	0.8906	0.8883	0.8859	4.7224	10.05	0.1075	0.1005	0.0953	False	0
43	2334.18	5.156362923365588e-06	0.9213	89.86	0.9014	0.8986	0.8963	4.7182	10.45	0.1142	0.1045	0.1011	True	0
44	2339.38	3.8060233744356633e-06	0.8999	90.75	0.9089	0.9075	0.9055	4.7272	9.81	0.1048	0.0981	0.0928	False	0
45	2356.84	2.653493525244721e-06	0.894	90.22	0.904	0.9022	0.9004	4.7515	10.29	0.1084	0.1029	0.0975	False	0
46	2348.93	1.70370868554659e-06	0.8782	91.46	0.917	0.9146	0.9131	4.75	9.65	0.1054	0.0965	0.0923	False	0
47	2359.35	9.607359798384785e-07	0.8728	91.23	0.9158	0.9123	0.911	4.7543	10.13	0.1128	0.1013	0.0975	False	0
48	2469.25	4.277569313094809e-07	0.8669	91.71	0.9197	0.9171	0.9153	4.7685	9.97	0.1076	0.0997	0.0954	False	0
49	2337.62	1.0705383806982606e-07	0.8644	91.64	0.9201	0.9164	0.9151	4.7535	9.89	0.1048	0.0989	0.0942	False	0
50	2580.46	0.0	0.8627	92.1	0.9239	0.921	0.9195	4.763	9.97	0.1057	0.0997	0.0948	False	0

Test (originals only, signer-disjoint, 312 clips) - BEST MODEL:

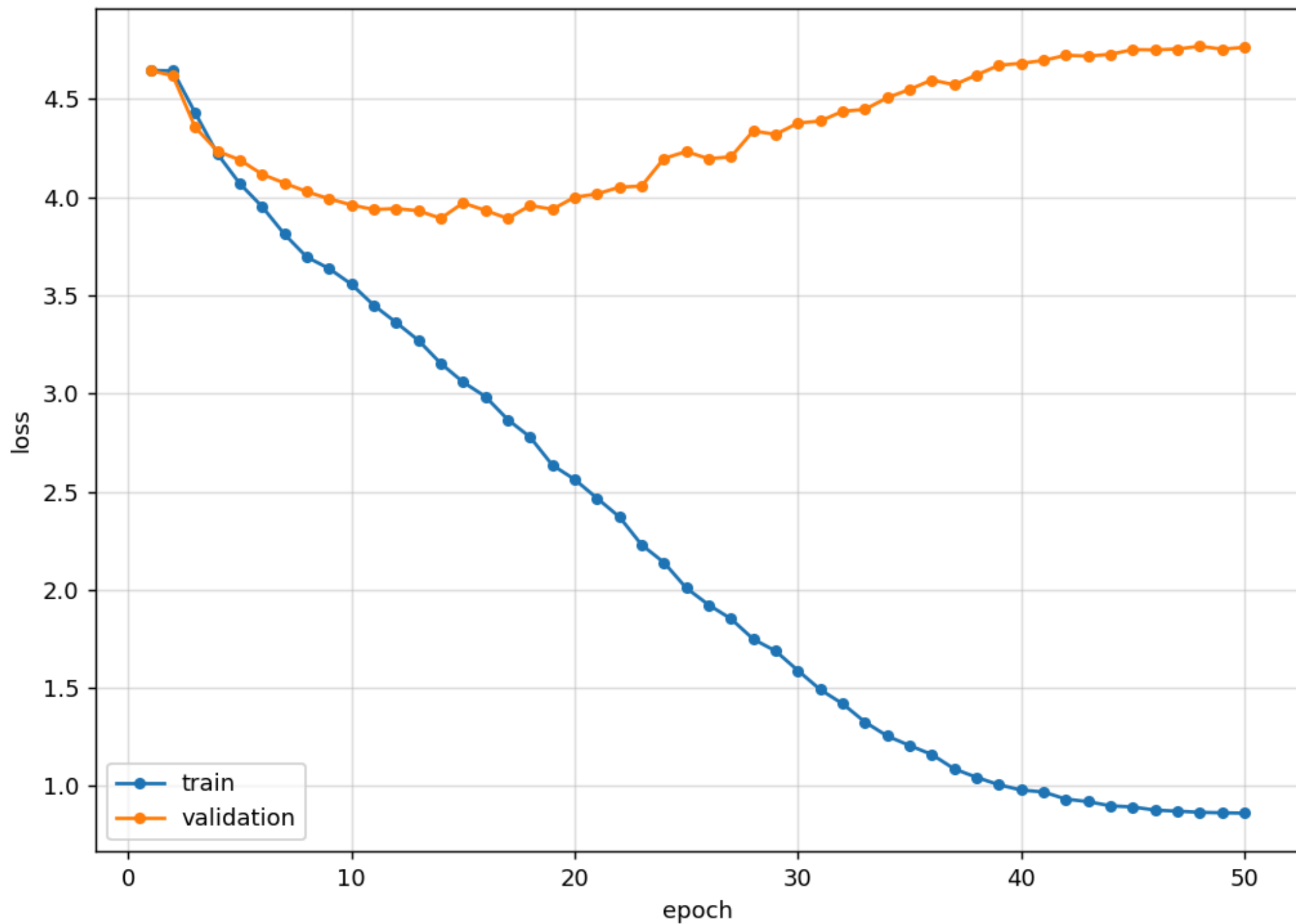
Metric	Value
test_loss	7.3092
test_top1_acc	2.24%
test_top3_acc	7.69%
test_top5_acc	10.58%
test_precision	0.0114
test_recall	0.0224
test_f1	0.0138

3. Read

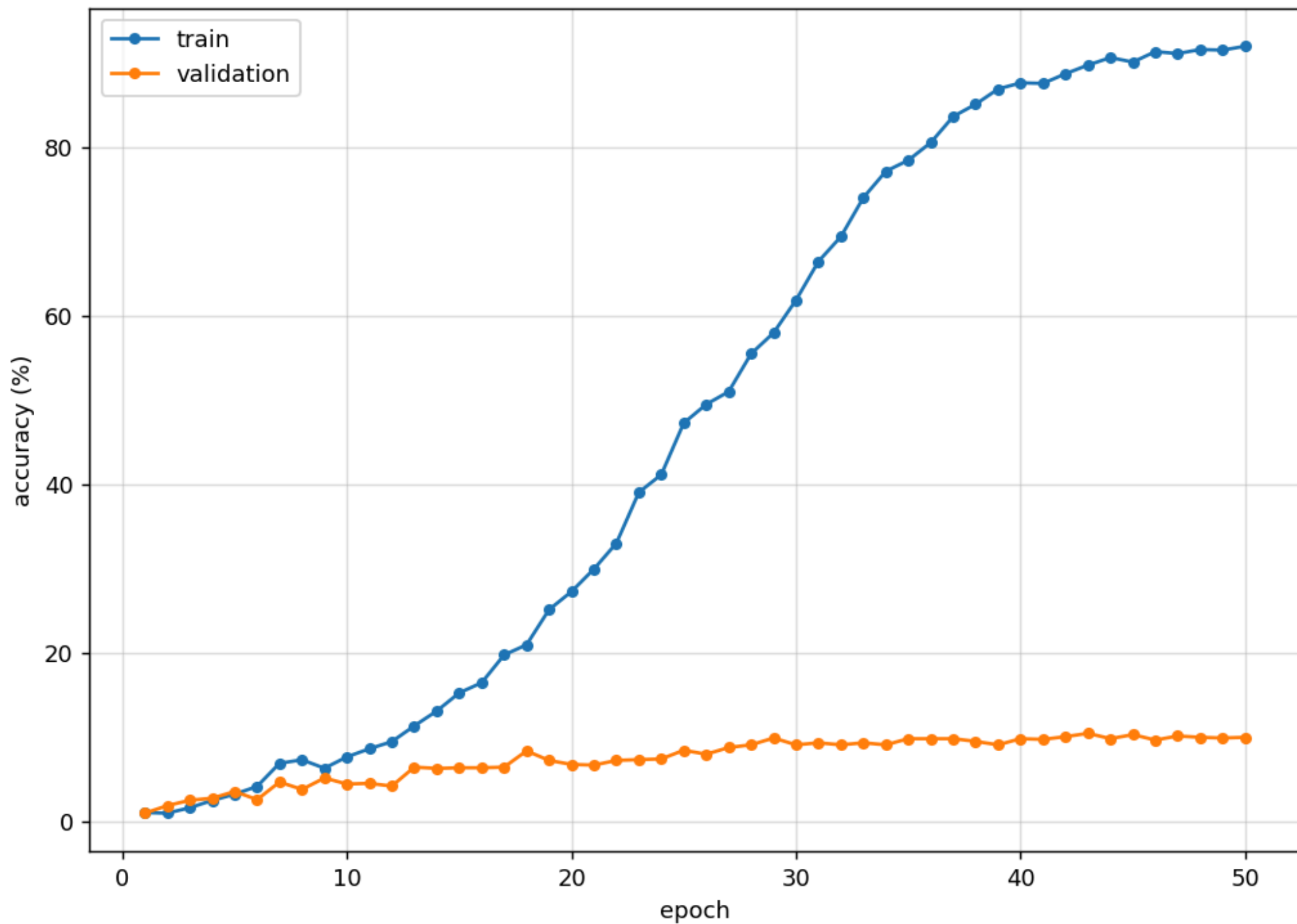
- Train converges cleanly (loss 4.65→0.86, acc 1%→92%). No architectural failure.
- Val plateaus at ~9–10% from epoch ~25 onward; "new bests" past that point are noise (9.08→9.32→10.29→...→10.45), not real improvement — early stopping (patience 15) never triggers because the counter keeps getting reset by marginal noise.
- Test top-1 (2.24%) sits *below* the entire val curve except the first ~2 epochs. Test loss (7.31) is far above final eval loss (4.76) — real distribution shift, not a bug (D9 assert confirms the eval pipeline is correct).
- Best usable framing for the resubmission: Top-5 accuracy (10.58%), consistent with the "signer-disjoint generalization gap" narrative, but weaker than the 92%→13% framing you've used before — this run's val ceiling itself is only ~10%, so the collapse is smaller in absolute terms but the qualitative story (train fits, val plateaus early, test is worse than val) still holds.
- **Handoff:**
 - Run A — Stage-1 style, signer-joint (older log)
 - Train/val/test = 4992/832/416, all randomly split from the same signer pool (no signer exclusion). Test acc hit 83.89% (epoch 10), then 82.21% after 2 more epochs. This number is not usable as a headline result — train, val, and test all share signers, so the model is partly just memorizing/interpolating people it's already seen, not proving it recognizes signs from a new person. Useful only as a "look how well it does when signer leaks" contrast baseline, if the paper wants to make that point explicitly.
 - Run B — Stage-5 recipe, signer-disjoint (the actual resubmission run)
 - Train/val (1248/412 orig, +aug) drawn from the same signer pool; test (312 orig) is from a genuinely different signer. This is the run that matches the paper's core claim (signer-disjoint generalization gap).
 - Result:
 - Train converges cleanly: loss 4.65→0.86, acc 1%→92%
 - Val plateaus at ~9–10% starting around epoch 25; every "new best" after that is noise, not real gains
 - Test: Top-1 2.24%, Top-3 7.69%, Top-5 10.58% — worse than the val plateau itself, and barely above chance (chance ≈0.96% for 104 classes)
- Why failed?
 - No inductive bias for signer-invariance anywhere in the architecture. Raw 3D convs + BiLSTM trained from scratch have nothing forcing them to separate "what sign is this" from "who's signing it." With this little data, the model takes the path of least resistance: it partially fits signer-specific appearance/motion signatures instead of the abstract sign class, because that's an easier function to fit at n=12.
 - Overfit onset is early and permanent. Val loss bottoms at epoch ~13-14, then climbs for the rest of the 50-epoch run while the train keeps improving. Everything past that point is the model getting better at Run A's problem (memorizing this signer pool) and worse at Run B's problem (generalizing). Patience=15 isn't catching it because "best val acc" wobbles up on noise even while the real trend is degrading.
- What could fix it:
 - a. Pretrained video/frame encoder, frozen or lightly fine-tuned. Swap the from-scratch 3D CNN trunk for frozen DINOv2/VideoMAE features. This is the single highest-leverage change: it imports invariances learned from millions of images/videos instead of demanding the model learn "hand vs identity" separation from 12 clips.
 - b. More signers in train, not more augmentation of the same signers. If at all possible, get train/val to include even 2-3 additional real signers rather than another augmentation pass. Synthetic transforms (rotate, hsv, speed) don't manufacture the one variable that's actually missing - a different person's motion.

- c. Explicit signer-invariance objective — e.g. domain-adversarial loss penalizing a auxiliary signer-identity classifier reading off the same features, forcing the backbone to discard identity-specific cues. Cheap to add on top of existing pipeline once encoder is swapped.

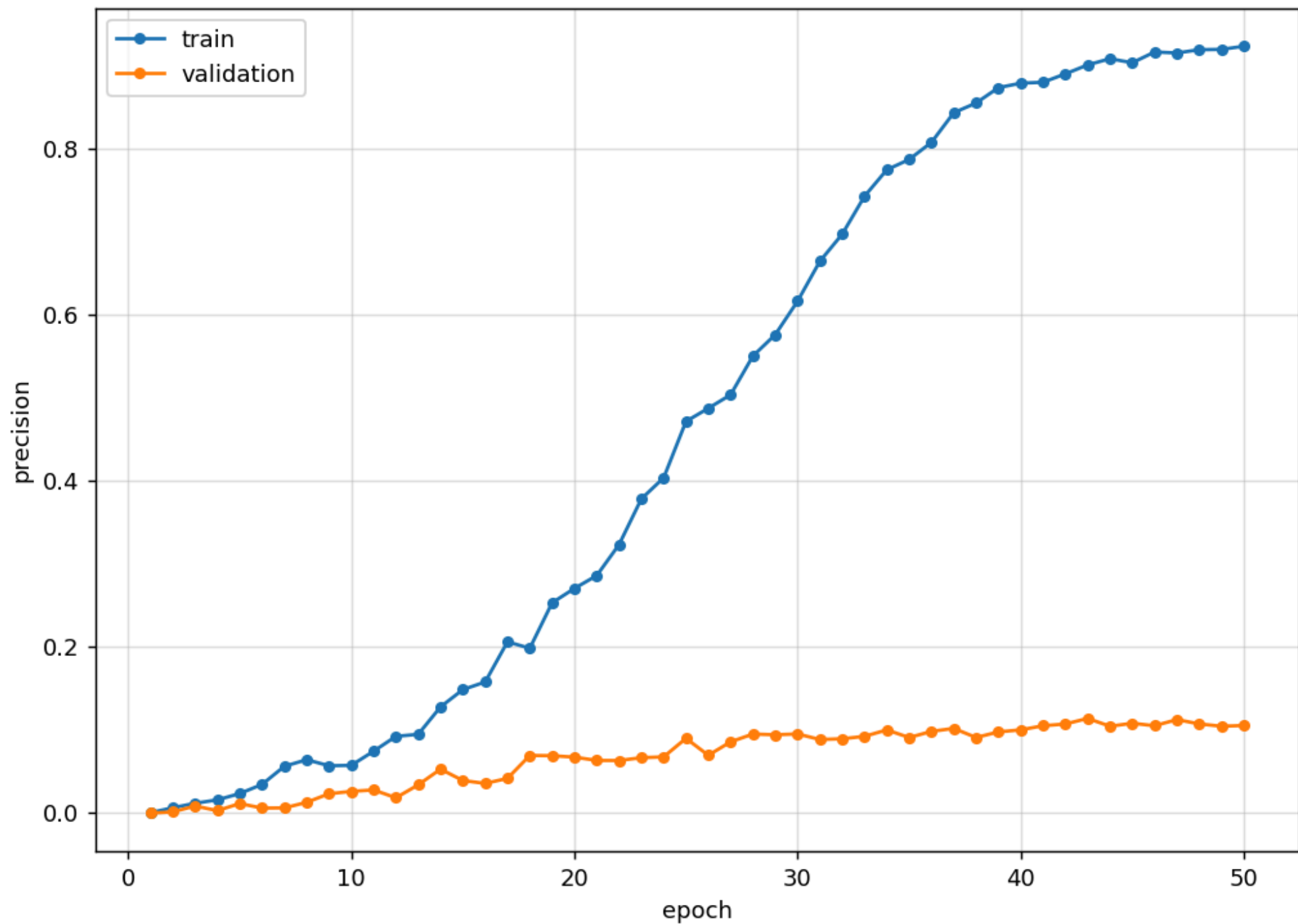
Loss per epoch



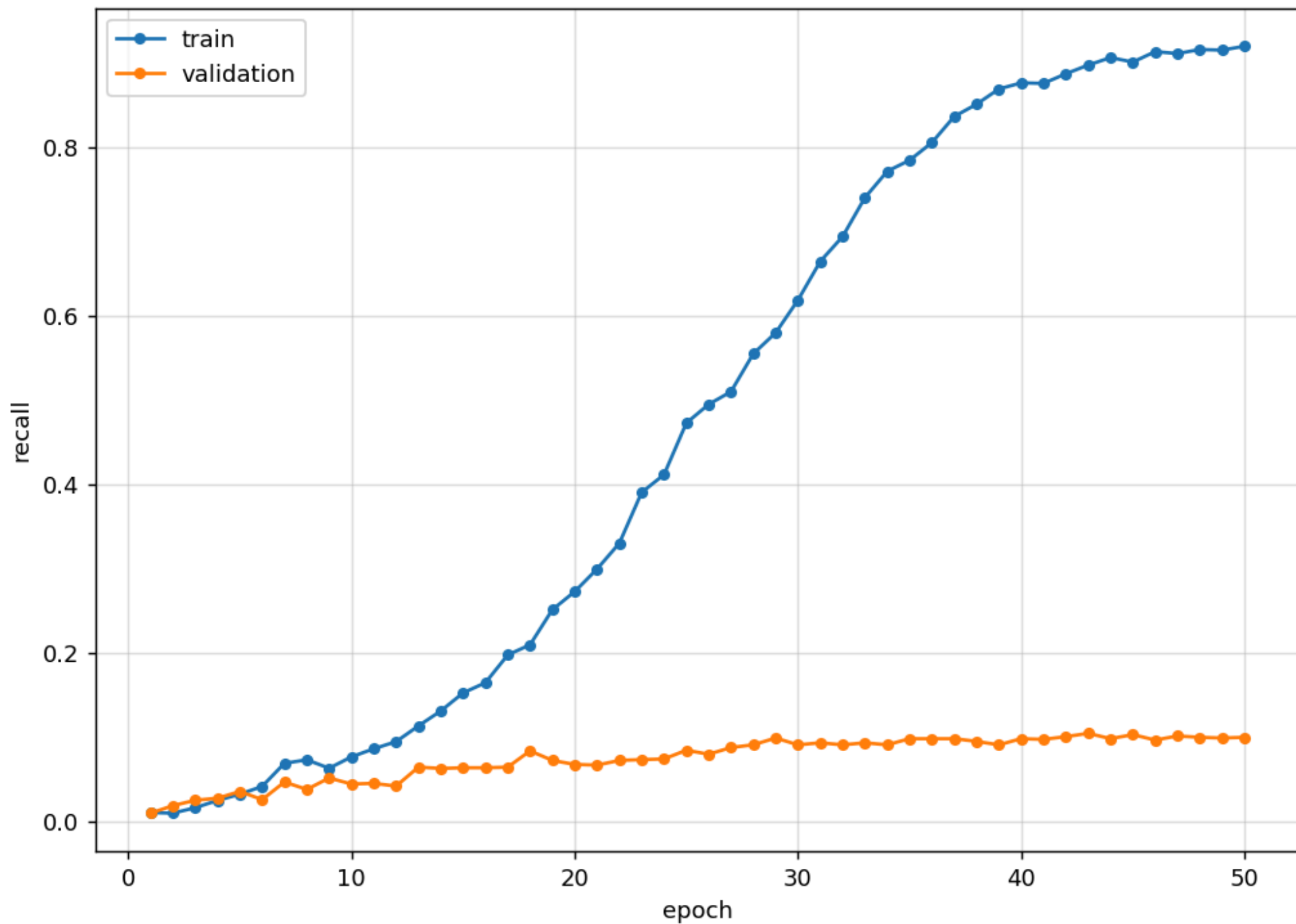
Accuracy per epoch



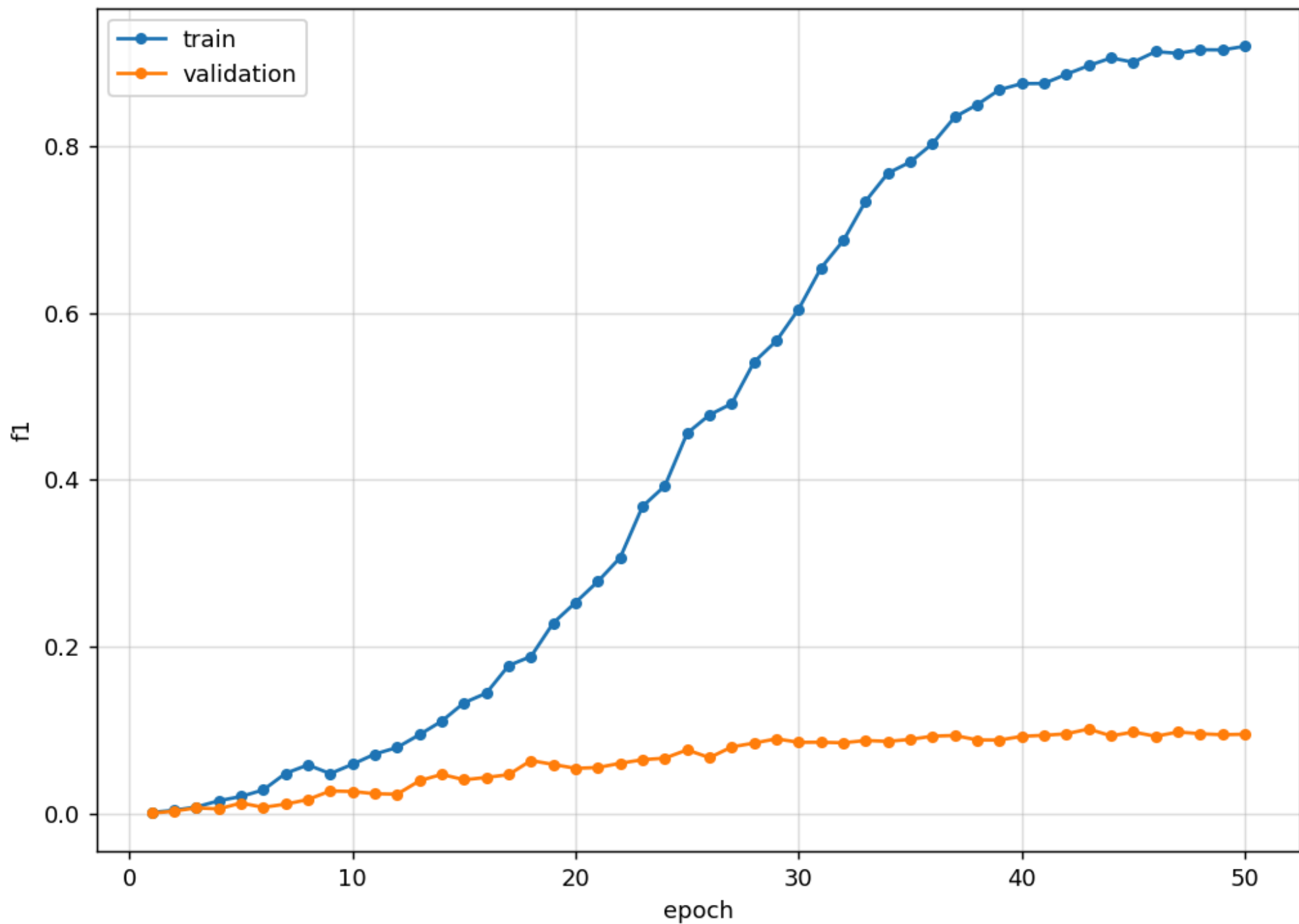
Precision per epoch



Recall per epoch



F1 score per epoch



Experiment 2

1. What Changed?

	V1	V2
BiLSTM layers	2	1
Post-LSTM dropout	0.3	0.5
weight_decay	1e-3	3e-3
hidden_size	576 (unchanged, output dim stays 1152)	576
Everything else (D1–D9: test-orig-only, sampling, L2 norm, Stage-5 recipe, early stop patience 15)	same	same

Hypothesis being tested: train/val gap in v1 was a capacity vs data mismatch (2-layer BiLSTM overfitting 12 clips/class) - cutting capacity + raising regularization should close the gap and push val/test ceiling higher.

2. Best Checkpoint V1 vs V2

	V1	V2
Best epoch	43	42
Best val accuracy	10.45%	11.01%
Train acc at best epoch	89.86%	63.94%
Train loss	0.9213	1.7733
Train precision	0.9014	0.6459
Train recall	0.8986	0.6394
Train F1	0.8963	0.6322
Val loss	4.7182	4.0071
Val precision	0.1142	0.1127

Val recall	0.1045	0.1101
Val F1	0.1011	0.1052

Worth flagging: v2's val loss is actually meaningfully lower (4.0071 vs 4.7182) at its best epoch. v2's best val acc is marginally higher, reached at a similar epoch, but at much lower train accuracy - exactly the signature capacity reduction should produce: less memorization for similar-or-better val performance.

3. Final test (signer-disjoint, 312 originals) - BEST MODEL

Metric	V1	V2	Δ
Test loss	7.3092	6.3687	V2 lower
Top-1	2.24%	1.60%	V2 worse
Top-3	7.69%	6.09%	V2 worse
Top-5	10.58%	9.29%	V2 worse
Precision	0.0114	0.0074	V2 worse
Recall	0.0224	0.0160	V2 worse
F1	0.0138	0.0094	V2 worse

4. Read

Val went up slightly (10.45%→11.01%). Test went down across every single metric (2.24%→1.60% top-1, 7.69%→6.09% top-3, 10.58%→9.29% top-5, precision 0.0114→0.0074, recall 0.0224→0.0160, F1 0.0138→0.0094). That's the **opposite of what a successful capacity fix should show**, and it's the most important result of this ablation.

The capacity cut worked exactly as intended, but only on val: v2 memorized much less (train acc 63.94% vs 89.86% at best epoch, train loss 1.77 vs 0.92) while matching or slightly exceeding v1 on val accuracy, precision, recall, and F1 - and v2's val loss is meaningfully lower throughout the back half of training (4.01 vs 4.72 at best epoch). **That confirms the "2-layer BiLSTM was overfitting" diagnosis was correct as far as val goes: less capacity, less overfitting to val, better-calibrated confidence on val.**

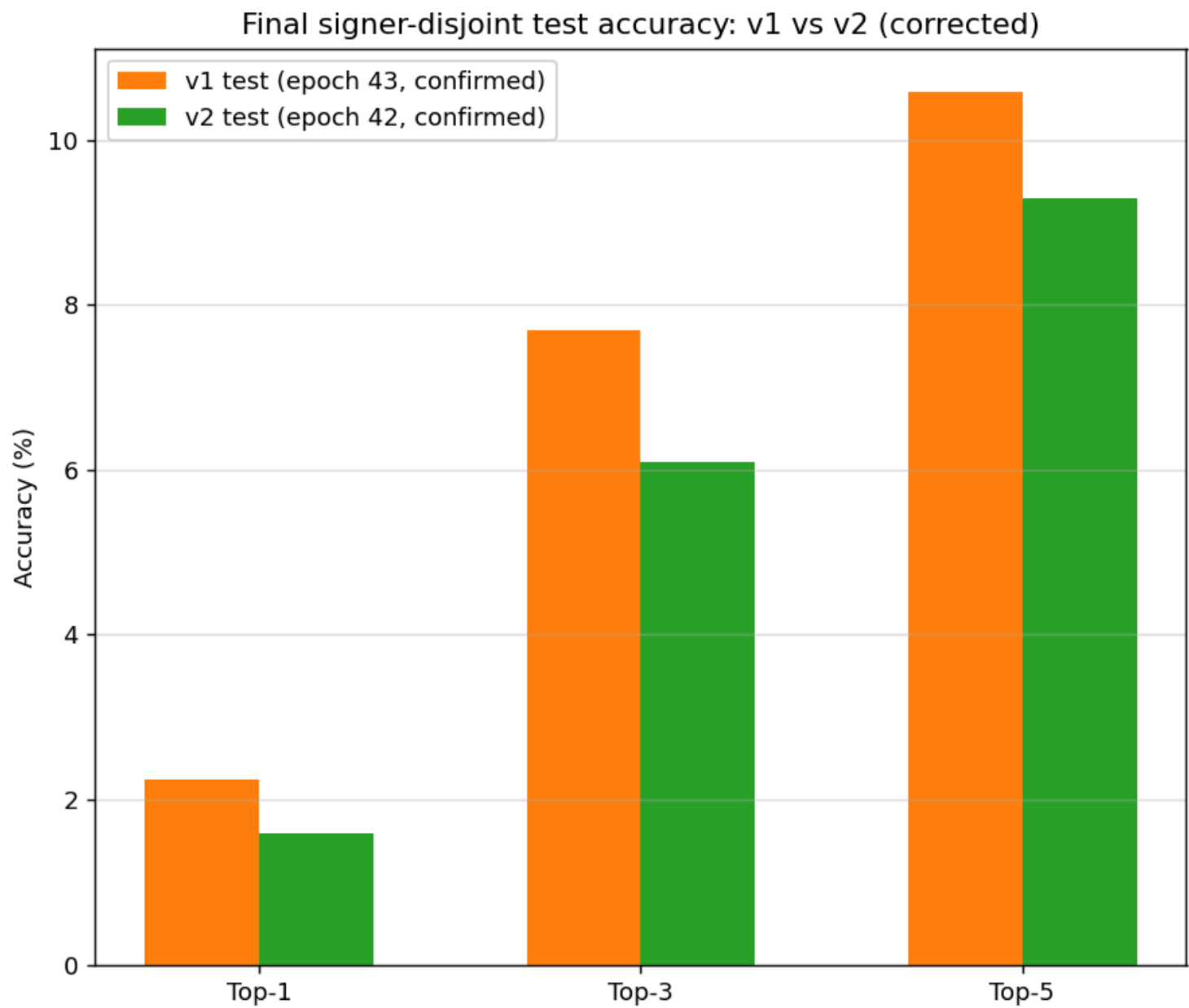
None of that transferred to the unseen test signer and it's worth being precise about why, because "it didn't generalize" undersells what's actually going on. The lower val loss reflects the model being less overconfident about wrong answers within the train/val signer pool specifically. That's a real, mechanistically-expected effect of more dropout and higher weight decay, but its calibration to a distribution the model has already seen variants of, not calibration to signs in general. The model was never shown a second signer during training, so it had no basis to learn "sign identity, independent of who's performing it." Cutting capacity removed its ability to memorize noise in the train/val pool; it did not, and structurally could not, teach it what a different person's motion looks like.

On test, this shows up as an unusual pattern: **v2's test loss is still lower than v1's (6.37 vs 7.31), but its accuracy, precision, recall, and F1 are all worse**. That combination means **v2 is appropriately less confident about predictions that are simply wrong more often, while v1 is overconfident about predictions that happen to be right slightly more often** - via memorization, not understanding. Neither model solved signer-invariance. v1 stumbled into marginally better test accuracy through brute-force overfitting; v2 has better-behaved uncertainty that still doesn't know the right answer for a person it's never seen.

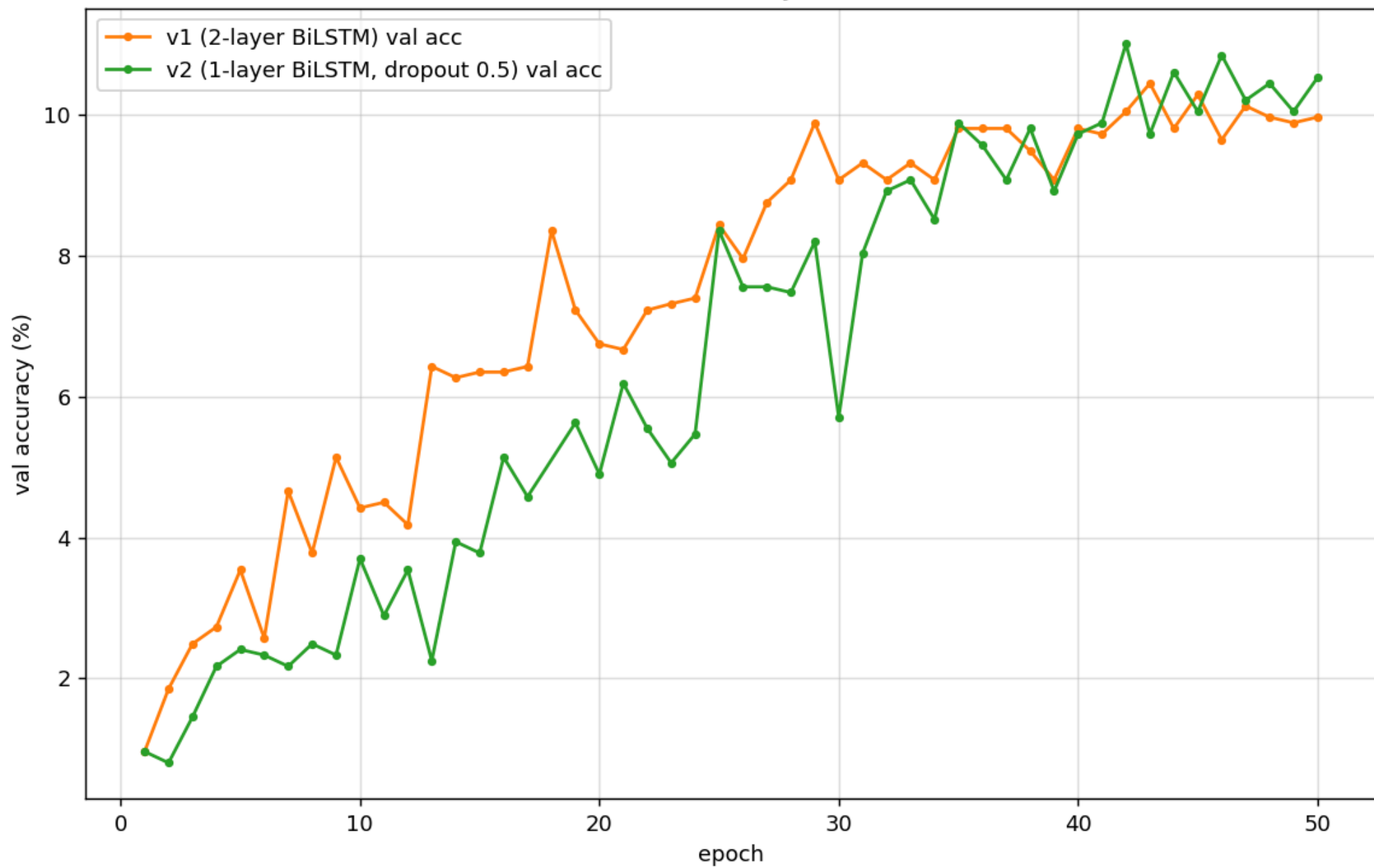
For the paper/handoff: frame this explicitly as a negative result that strengthens the argument. "We tested whether the signer-disjoint generalization gap was a capacity artifact by halving the BiLSTM depth and increasing regularization (dropout 0.3→0.5, weight_decay $1e-3 \rightarrow 3e-3$). The modified model showed the expected signature of reduced overfitting: lower train accuracy, lower validation loss, comparable validation accuracy, but test performance on the unseen signer degraded across every metric (top-1, top-3, top-5, precision, recall, F1).

This indicates the **collapse reflects the absence of signer-diverse training data rather than excess model capacity** - architecture tuning within the current from-scratch design has been ruled out as the primary lever. The next experiment with real expected payoff is swapping the from-scratch 3D CNN trunk for a frozen pretrained video/frame encoder (e.g. DINOv2 or VideoMAE features), ideally combined with adding real additional signers to train/val rather than further augmentation of the existing 12

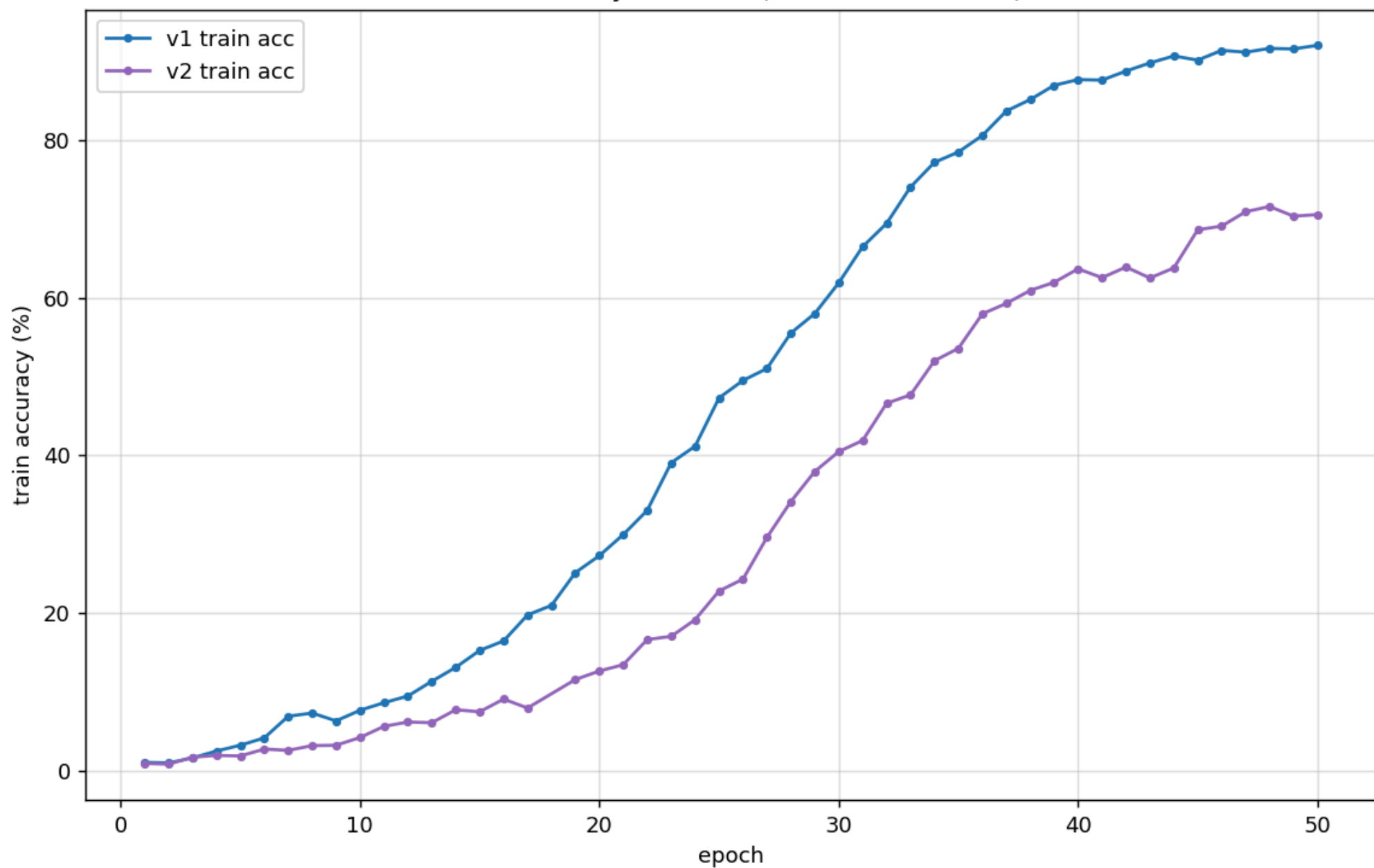
4. V1 vs V2 charts



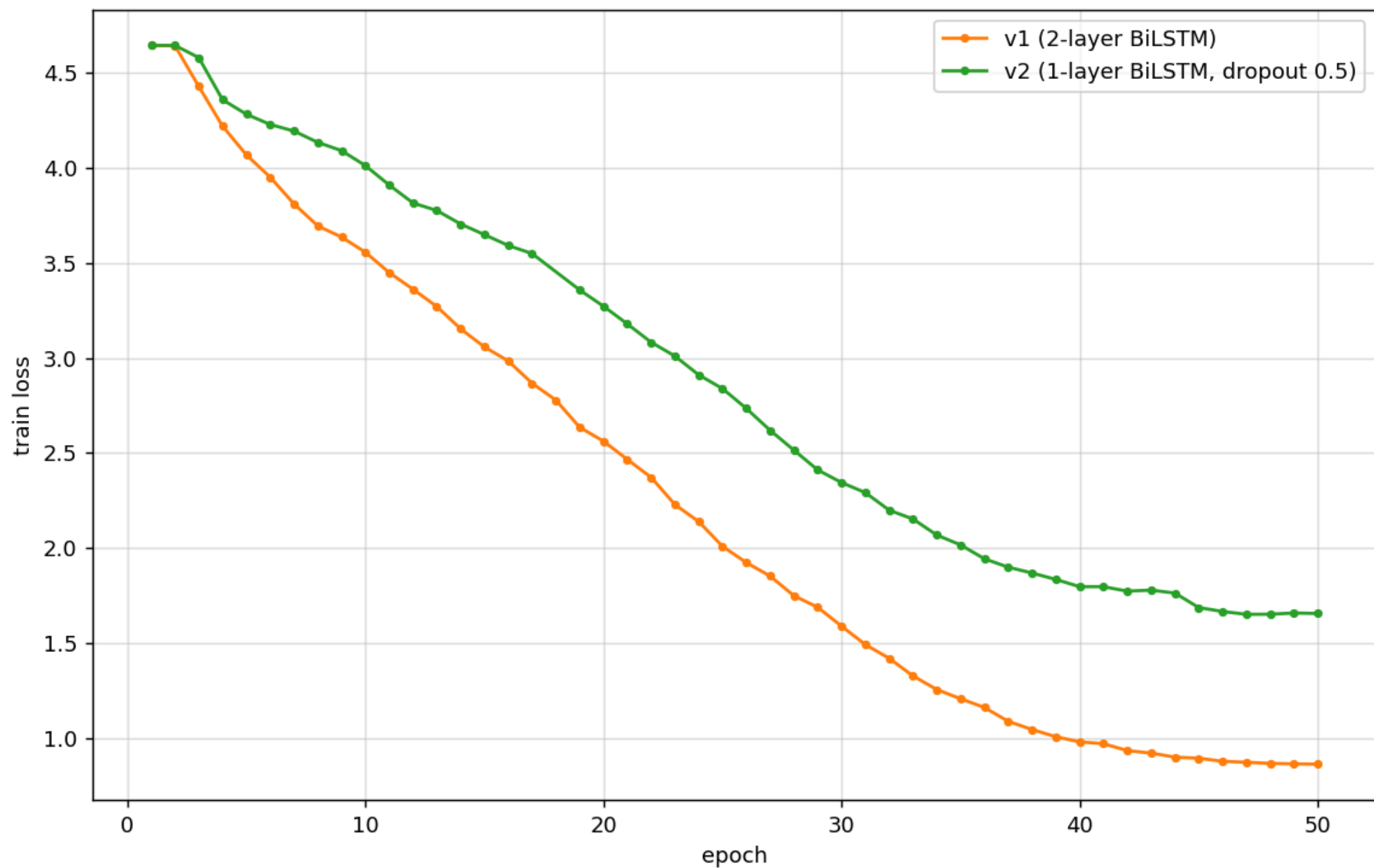
Validation accuracy: v1 vs v2



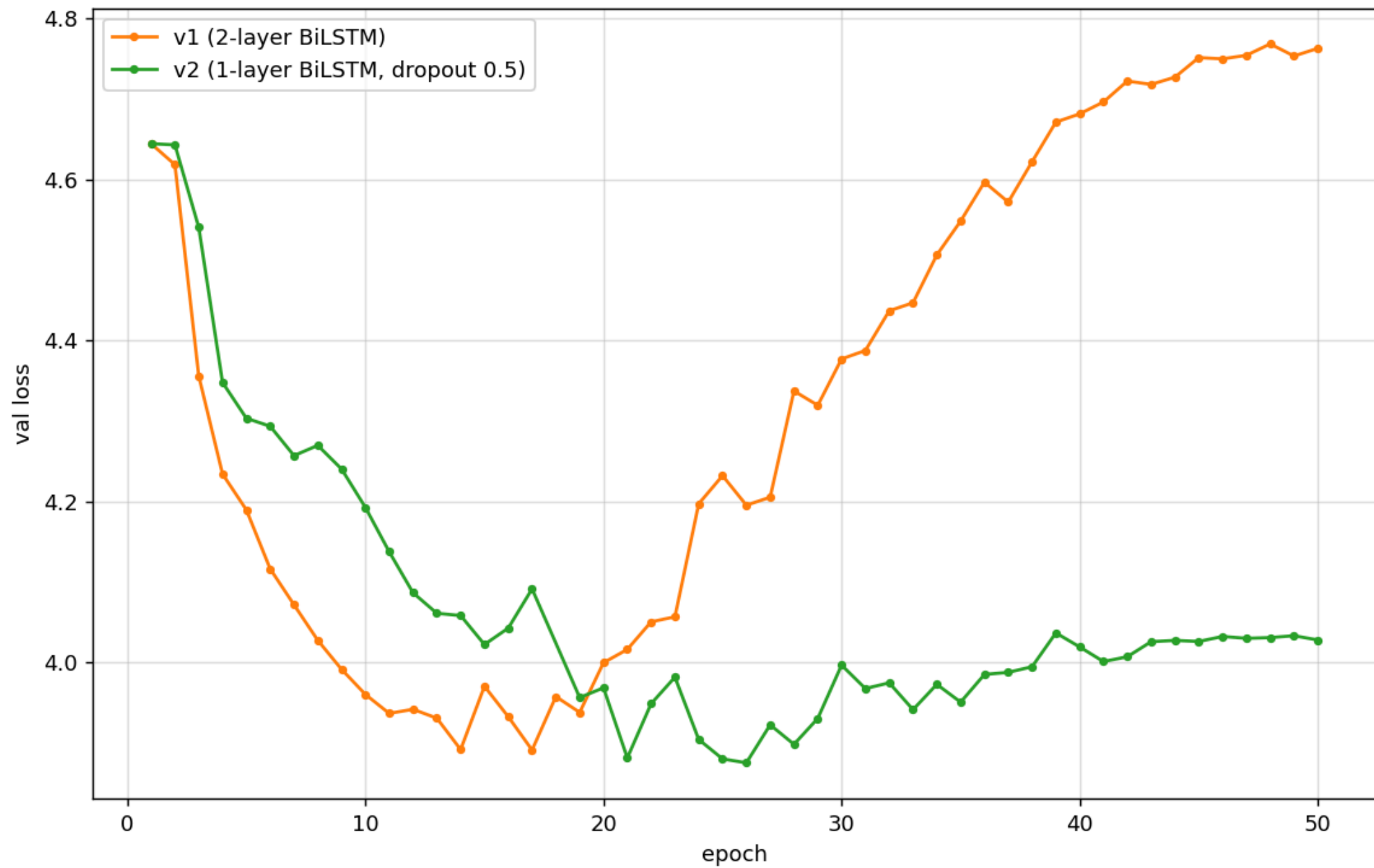
Train accuracy: v1 vs v2 (v2 memorizes less)



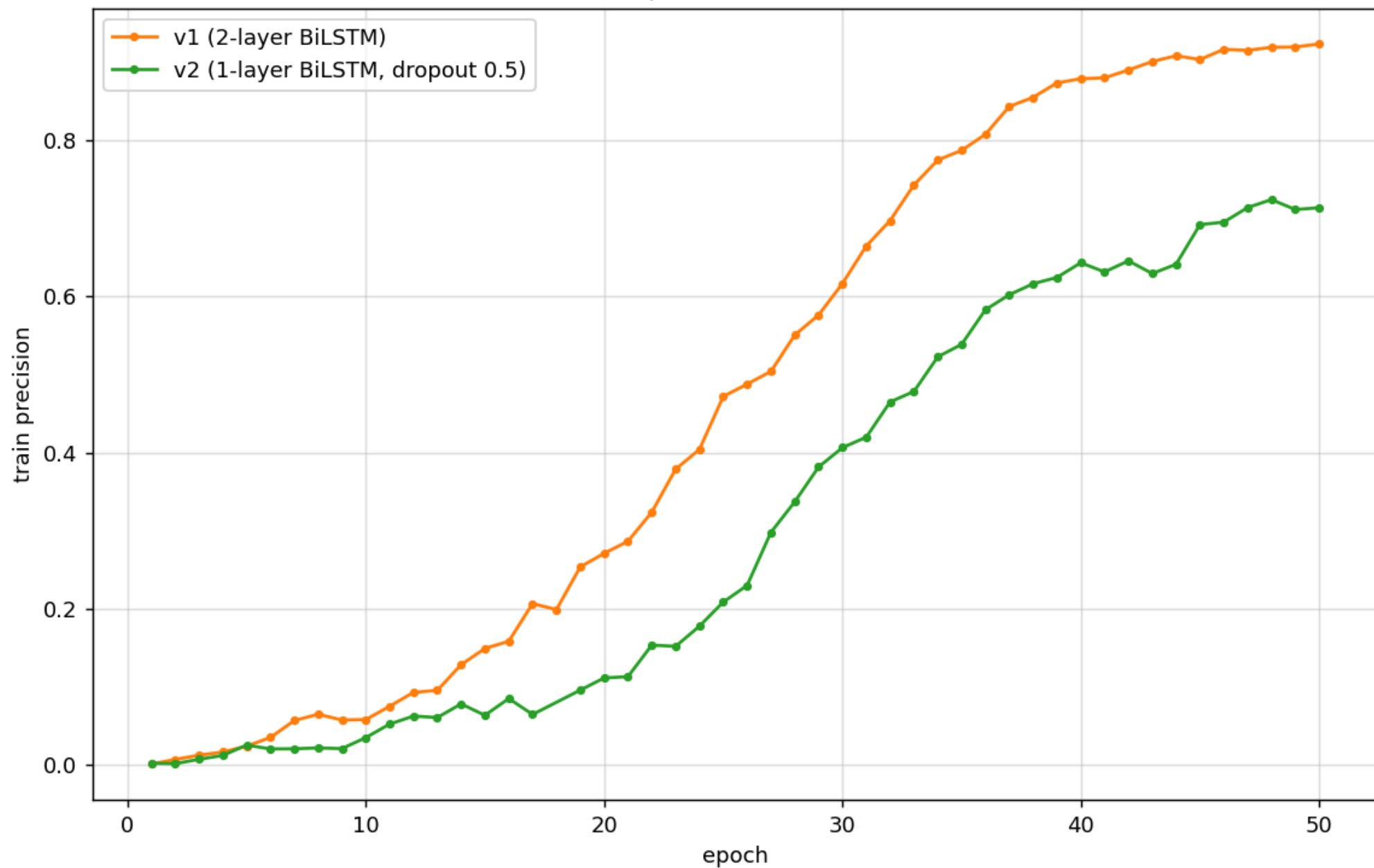
Train loss: v1 vs v2



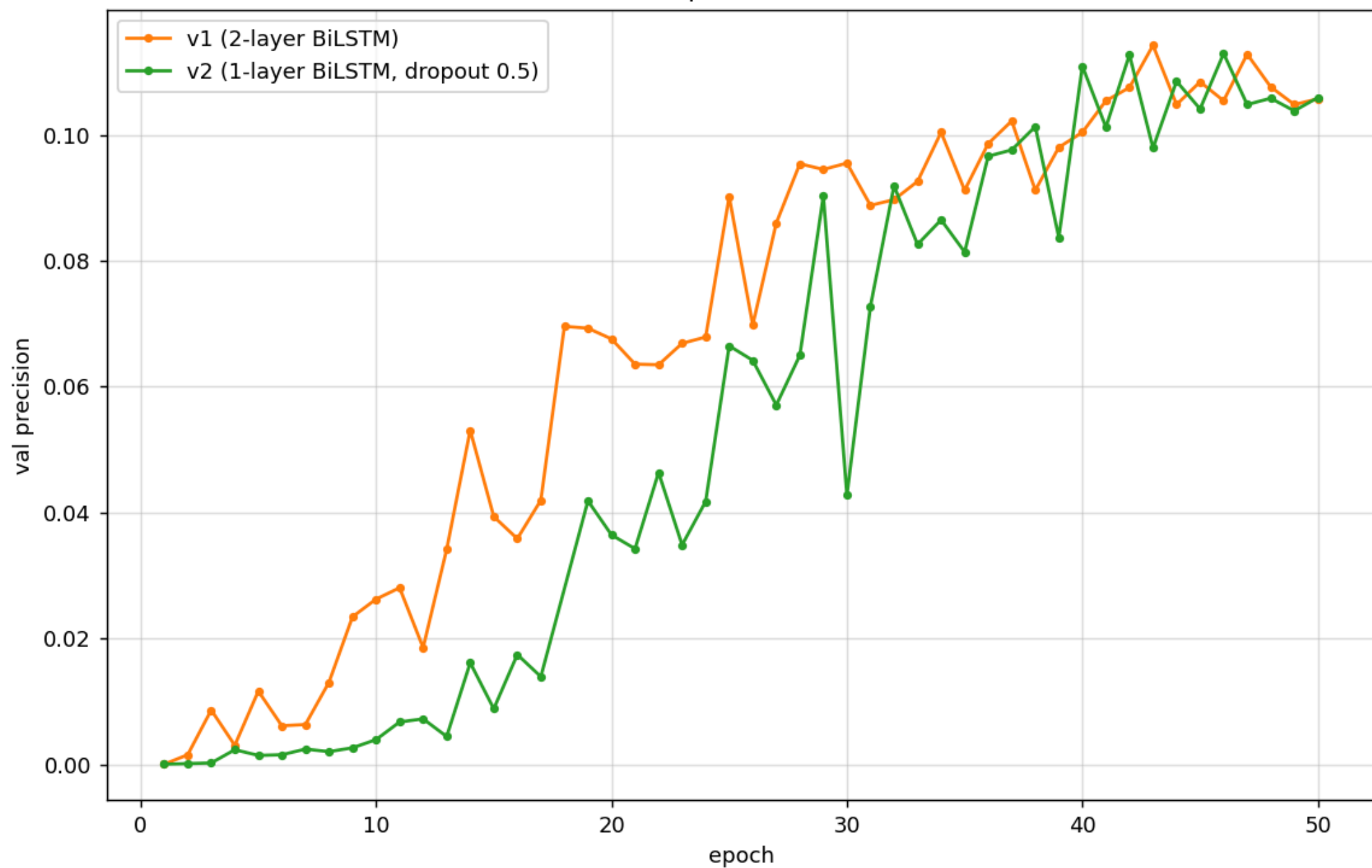
Validation loss: v1 vs v2



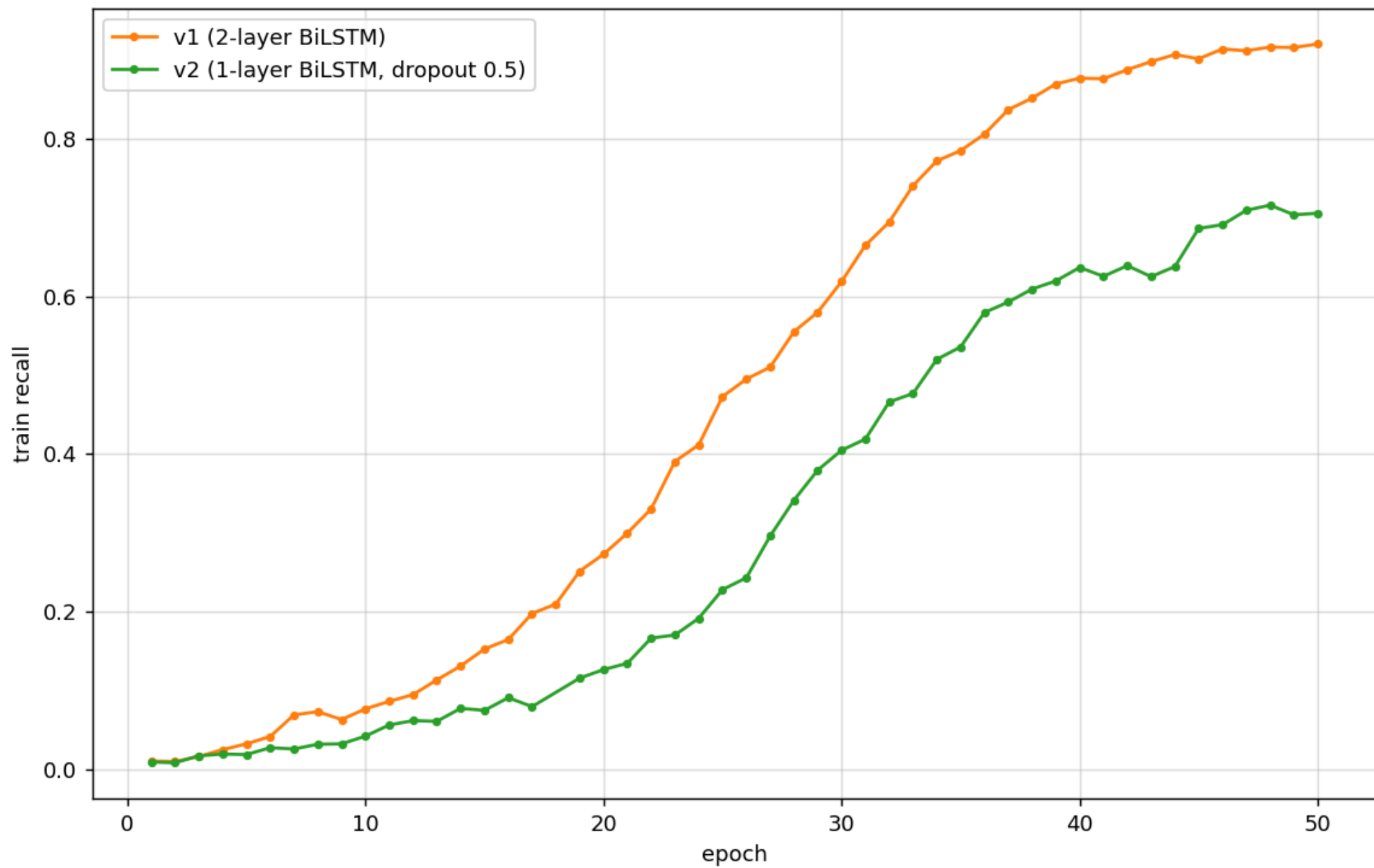
Train precision: v1 vs v2



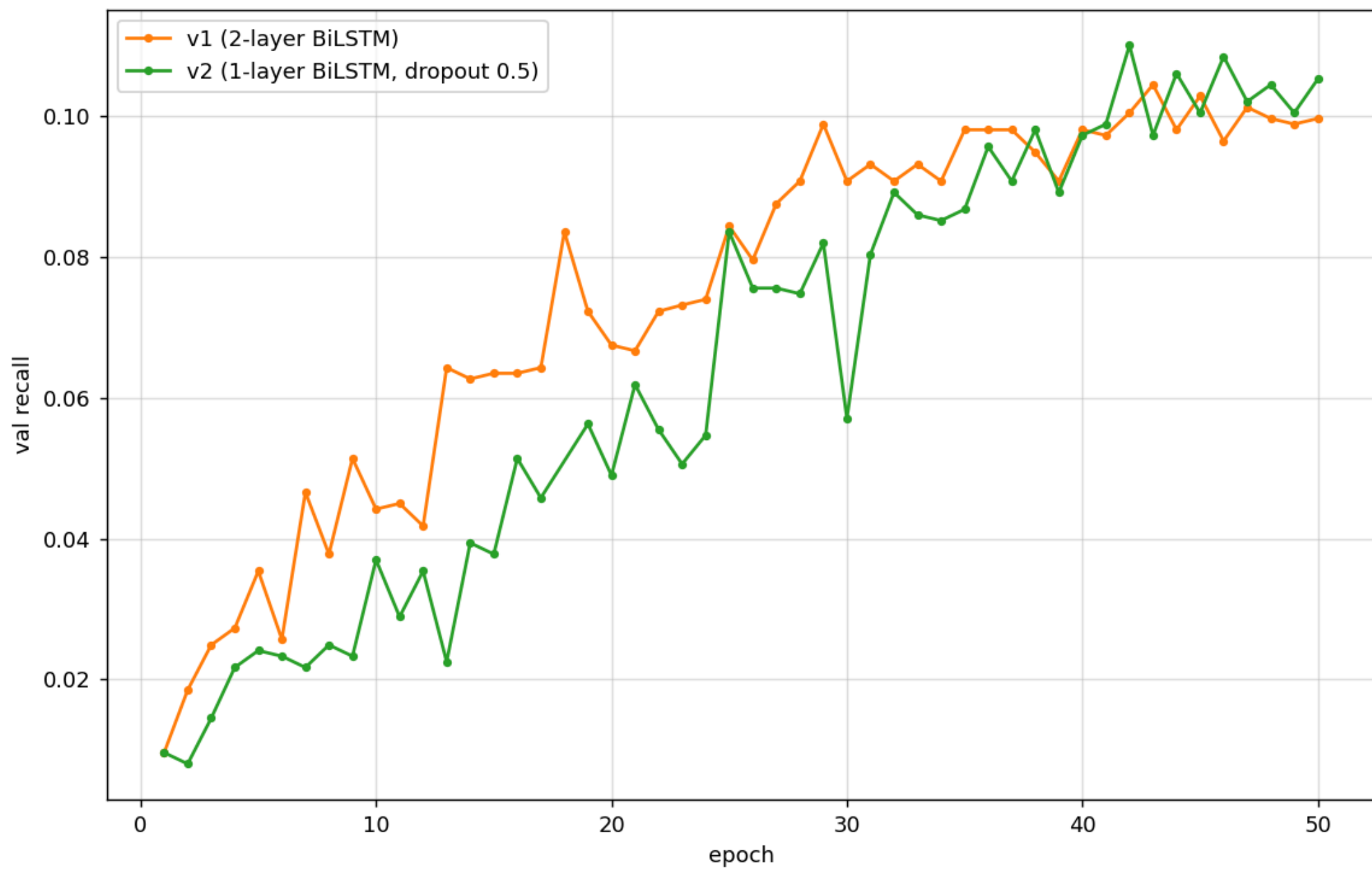
Validation precision: v1 vs v2



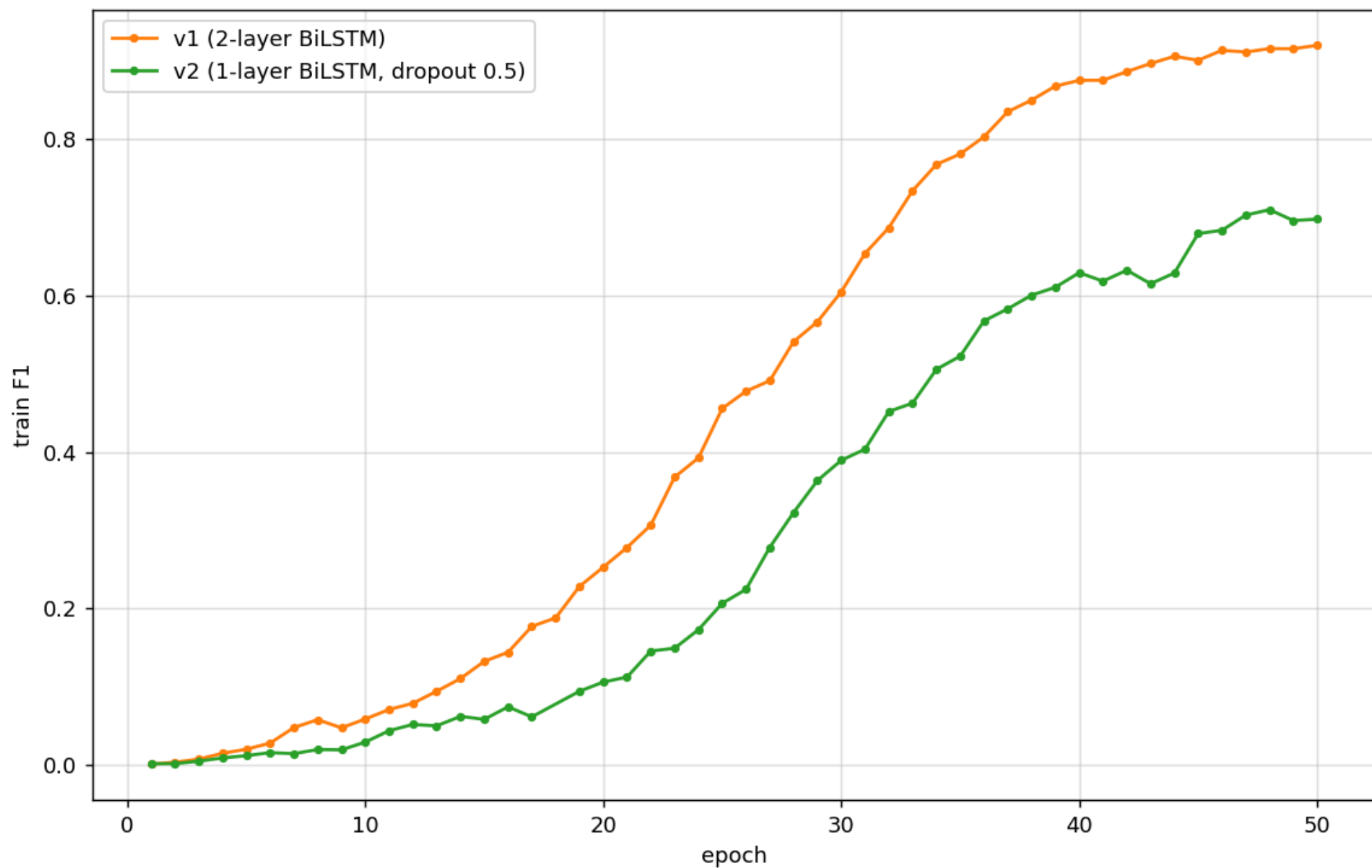
Train recall: v1 vs v2



Validation recall: v1 vs v2



Train F1: v1 vs v2



Validation F1: v1 vs v2

